Recursive self-improvement, gated by evidence

An engine that lets deployed agents propose, test, canary and ship corrections to their own operating knowledge, measured where the world changes under them

SenseLab Research

`https://sense-lab.ai`

Preprint v0.3, October 2026

Abstract

An agent that runs for months inherits a world that changes under it. Policies are rewritten, dependencies re-pinned, workflows changed, and nobody tells the agent, so it acts on what used to be true and every later session that reads the same thing fails the same way. Scaling such agents is a self-improvement problem: the system has to notice that its knowledge is wrong, draft a correction, test it, release it to a small share of traffic first, and keep a record, without a person in every loop. SenseLab is an engine built for that. Outcome evidence changes what agents act on (level 0); a repair agent built into the engine drafts corrections that must pass a stand-in replay, the customer's own replay and a live canary before they ship, reversibly (level 1); the loop reads its own history when it drafts again (level 2); and its record trains the next model and the repair prompt (level 3). This paper is the first measurement of such a loop. We evaluate the whole engine on a continual benchmark where the correct action for a task class changes conditionally at episode 30 of 100, so half the class needs the new rule and half still needs the old one. On four held-out seeds, two agent models and two task families (12,800 episodes, analysis frozen before the run), SenseLab takes the new action first in 45 % of moved-half episodes, against 25 % for the frontier model alone, 32–36 % for three pgvector baselines and 40 % for a 24k-token context window; it wins 8–11 of 16 paired cells against each baseline, and no pairing survives Holm correction. It holds unchanged work at 100 % and loses the kept half to the context window by 24 points at a third of the cost. In ten-agent, 200-episode cells the moved half rises from 37 % in the first fifty post-change episodes to 75 % in the last fifty. In an exploratory 400-episode run with the traffic the canary rule is sized for, five of eight canaries decided: four passed and shipped, and one wrong fix failed at 88 % recurrence against 10 % before it reached `main`. Along the way the benchmark found twelve defects in the loop's evidence path, all fixed in production and re-measured on the same protocol. That record is the paper's main contribution: a self-improving system is only as trustworthy as its evidence path, and a retrieval benchmark would have seen none of these.

1 Introduction

The hard part of scaling agents is longevity. A model that solves a task today is deployed into an operation that runs for months. The policies it applies get rewritten, the dependencies it pins get audited, the workflows it follows get changed by people who never tell it. Its tasks never repeat exactly, so nothing can be memorised. The failure that matters is that it acts, confidently, on what used to be true, and does so again on every session that reads the same thing. A static model with a static context degrades in such a world, and a person in every loop does not scale.

The alternative is a system that improves its own operation from its own results, which is also the thing to be nervous about. A self-modifying system amplifies its errors as readily as its corrections. A model that drafts fixes from failure evidence will draft wrong ones, and a wrong fix

written into what every agent reads damages every session at once. The question is what discipline makes self-improvement safe enough for production: evidence before proposal, proof before release, a small exposure before full exposure, reversibility after, and an audit trail throughout.

SenseLab is an engine built on that discipline, and this paper measures it. Its loop has four levels (§2). At level 0, every session's outcome is attributed to exactly what the agent read and did, and that evidence changes what the next session acts on. At level 1, an entry that was validated and now fails is a *regime shift*; the engine's repair agent drafts a correction on a branch, with a manifest of the runs it claims to repair and the runs it must not break, and the correction must pass a stand-in replay (Tier 1), the customer's own agent replaying the case set (Tier 2) and a live canary on a hashed share of sessions (Tier 3) before it ships, reversibly. At level 2, the loop's history on an entry is evidence for its next draft. At level 3, the record trains the next model and the repair prompt. The claim is that a correction may only change behaviour after it has been tested on the runs it claims to repair and shown no worse than the control arm on live traffic, and that a system built this way recovers faster than one that can only demote what failed, while shipping fewer wrong corrections than one that ships on a passed test.

We ask, on a benchmark that knows the true rule before and after every change, whether the whole engine recovers faster and further than the frontier model alone, than three vector-store baselines, than a context window carrying the cell's entire history, and than its own ablations; whether it damages what still worked; what it ships; and what it costs. We call the whole system SenseLab throughout. The ablations that switch parts of it off appear in one attribution table and nowhere else.

What this version is. This is the first version with the confirmatory read: seeds 37, 41, 59 and 73, both agent models, run once after the analysis code was tagged, with every endpoint and test fixed beforehand (§5). The development-seed sections (§4) are kept because the preregistration committed to reporting every prediction, met or falsified. Most of the early predictions were falsified, by defects in the loop's evidence path that the benchmark found and that were fixed in production between runs; §6 is the record of that. The fleet-1 confirmatory cells are read and final. The ten-agent fleet-10 cells ran twice, because the first run exposed the last of the findings, and the second run is the read reported here.

2 The engine under test

Everything measured is production code on the SenseLab SaaS, run against a dedicated Enterprise account. No constant below was tuned for this study; each is quoted from the deployed source. The harness never decides anything. It seals sessions through the public API, and the loop's own scheduler proposes, tests, gates and ships.

Level 0: outcome learning. An agent calls `briefing` and `retrieve` before acting, `record_action` and `record_attempt` as it goes, and `commit_outcome` when the task ends. The sealed trace carries the exact read set, and the outcome is attributed to it: an entry read by a run that failed after a recorded attempt against it is charged the failure, and an entry read by a clean success is credited. Entries carry an evidence status (`untested`, `validated`, `contested`, `discredited`). `retrieve` returns, with the hits, per-action priors (won/lost here on similar tasks) and a recommendation mode: `act`, `explore` (assigned per agent, so peers explore different actions) or `escalate`. Level 0 alone is the row "SenseLab − repair loop" in Table 4.

Regime shift and situation. A regime shift is an entry validated at least three times that now fails: a sealed failure whose read set contains such an entry, or a recorded failed attempt against it. An entry that was never validated cannot have shifted; the guard that separates the two cases was the third defect this study found (§6). The same entry can be right in one situation and wrong in another, so the agent labels each session with an `attributes.situation`, and the service resolves the label to the agent's canonical situation (a stemmed-token rule, Jaccard ≥ 0.5, with an embedding second stage at cosine ≥ 0.75). The shift, the proof set and the canary are all scoped to the situation. A shift proposes a fix automatically (`auto_propose`, per agent).

Level 1: proposal. SenseLab's repair agent, a platform-side component of the engine with a 50k-token budget per draft, reads the failing runs, the entry and the loop's history on that entry, and writes either a corrective entry (lever `knowledge`) or a conditional procedure with conditions, steps and a verify (lever `procedure`) on memory branch `repair/<fix>`. It attaches a manifest: the runs it predicts it repairs, the runs it must not break, and an expected prevention (0.5 by default). A `code` lever is handed off and never auto-shipped.

Tier 1: stand-in replay. A stand-in model re-runs each manifest case with the branch as memory and tool calls answered from the recorded run. Fidelity is the share of steps answered from the record; a test below the agent's fidelity floor (0.5) does not count as a pass. A test is `passed`, `failed` or `inconclusive`.

Tier 2: customer replay. The fix's case set (HMAC-signed cases) is delivered to the customer's agent, which re-runs them on the branch and seals the results; the service grades each replay against the case's expectation. For a regime fix the case set is the run the shift fired on, up to three more of the situation's failed runs (the fix must turn them into successes) and up to three of its passed runs (the fix must not break them). A case no judge graded is graded on the outcome the replay reports and, since the `post-clean-success` cut, on its attempts: a success reached after a recorded failed attempt fails the case (finding 9).

Tier 3: canary. A passed fix opens a canary. Sessions are bucketed by sha256(*fix*:*session*); a share `pct` reads from the branch, writes always go to `main`, and the trace is stamped with its branch. `decide()` requires both arms to reach `min_outcomes` graded runs, where a graded run is one in the fix's situation or one that read an entry the fix is about (cut `post-canary-ni`; before it, situation only: finding 11). The canary is the loop's harm gate. The replay tiers prove the benefit on the failing runs; live traffic has to show the change does not make the agent worse on the runs the old rule still served. It passes when the canary arm recurs significantly less (one-sided two-proportion z-test, $p < 0.05$), or when the one-sided 95 % Newcombe upper bound on canary−control recurrence is under a 10-point margin with the failure still live in control. It fails when the canary arm recurs significantly more. Before `post-canary-ni` the primary asked for a significant improvement, and no canary ever passed. The benchmark maps the production defaults (10 %, 20 outcomes, 14 days) to `pct`=50, `min_outcomes`=8 and a timeout of 60 sessions (the cell horizon, 100 or 200, from amendment (v) on), evaluated every 5 graded sessions. One canary per agent at a time.

Ship, verify, revert. Policy `auto_after_canary` merges a passed canary's branch into `main`; `auto_after_replay` (the "− canary" ablation) merges on a passed Tier 2. A feedback contract holds a fix if fewer than 80 % of recent traces carry provenance. Prevention is verified over the

following window, and a regressed fix is rolled back to the pre-fix versions. Continuous judges (a two-tier `gpt-5-mini`→`gpt-5` rubric pack) grade sealed traffic and open incidents, the second producer of proposals.

Level 2: the loop reads its own history. Every fix event on an entry (proposed, tested, replayed, shipped, regressed, rolled back) is written back as evidence on that entry, and the repair agent's next draft on the same entry reads it. Whether the loop stops repeating its own mistakes is the two-change protocol of the flagship experiment, not yet run.

Level 3: the record trains the model and the improver. The traces the loop produces, including the repair trajectories and the cases it pinned, are a training set. Managed Models (`senselab-s1`) are distilled from them and served with the same evidence record as their memory (§4). The repair prompt itself is A/B tested across proposals, scored by proven fixes per prompt. Only the first rung of the first of these is measured in this version.

3 Benchmark and protocol

Preregistration. Hypotheses, arms, metrics, seeds, the dev/confirmatory split and the analysis freeze were registered before the first run (`HYPOTHESIS_RSI.md`, tag `prereg-rsi-v1`). Every change since is a dated amendment, (a) through (ab), appended below the original with a prediction and what would falsify it. Seeds 11 and 23 are the development seeds; 37, 41, 59 and 73 run once, after the analysis code is tagged `analysis-freeze-rsi`. Engine changes made between runs are recorded with their deploy cut to the second, and every result carries its cut.

Environment. Two of the benchmark's ten task families, both with tool-using agents and oracle-graded outcomes: `support` (a helpdesk agent with account, order and export tools; 15 issue classes) and `ci-fix` (an agent repairing failing CI on a repository; 10 failure classes). Every task instance is generated, never repeated, with random identifiers (`opaque_ids`) so nothing can be read off an id. 100 episodes per cell, 300 distractor entries in every store, retry budget 3, one agent identity per cell, GPT-5.5 at low effort, and a post-episode reflection note in every arm that has a store.

Change set C. At episode 30 a subset of classes changes conditionally: the correct action moves for instances with one attribute value (the *moved half*) and stays for the others (the *kept half*). Half of post-change tasks come from changed classes. Set C was written after the pilots showed that on an unconditional change (set A) outcome learning alone recovers nearly everything a maintainer who knew the rule would (finding 4). The only place a written rule can beat demotion is where demotion has no signal, because the entry is still right half the time. Set C is registered by hash; its remaining eight families are written blind.

Systems. Frontier alone (no store); context window (every prior note and outcome of the cell in the prompt, most recent first, 24k tokens); pgvector (bge-small, cosine top-5 over notes); pgvector + rerank (the cross-encoder SenseLab serves); pgvector + outcomes (notes, LLM consolidation every 5 episodes, a per-entry win/loss counter shown to the agent); and SenseLab (§2, policy `auto_after_canary`). Mem0 and Zep are not run; an earlier grid covers them [1]. The ablations, SenseLab − canary (`auto_after_replay`) and SenseLab − repair loop (policy `review`, proposals never approved), appear in Table 4 only.

Endpoints. First-attempt success on the moved half after the change (primary); first-attempt success on the kept half and on unchanged classes (collateral damage); episode success and escalation; the fix lifecycle (proposed, Tier 1, Tier 2, canary, shipped, reverted); and cost. Infrastructure errors are excluded from rates and reported. Intervals are cell-level bootstrap (2,000 resamples). With four cells per system on the development seeds they are wide, and they are shown so that no reader mistakes a dev-seed difference for a result.

4 Results on the development seeds

4.1 After the rule changes conditionally

system	moved half	kept half	unchanged classes	pre-change	episode success	escalated	$/ep
Frontier alone	17 % (13/75) [0, 41]	38 % (25/65) [11, 78]	66 % (93/140)	61 % (73/120)	84 % (334/400)	16 % (66/400)	0.011
Context window	33 % (25/75) [0, 78]	98 % (64/65) [94, 100]	100 % (140/140)	85 % (102/120)	92 % (366/400)	8 % (34/400)	0.070
pgvector	17 % (13/75) [2, 38]	45 % (29/65) [13, 78]	93 % (130/140)	71 % (85/120)	88 % (353/400)	12 % (47/400)	0.027
pgvector + rerank	5 % (4/75) [1, 9]	86 % (56/65) [78, 95]	94 % (132/140)	79 % (95/120)	90 % (359/400)	10 % (41/400)	0.028
pgvector + outcomes (DIY)	19 % (14/75) [0, 44]	55 % (36/65) [39, 78]	80 % (112/140)	70 % (84/120)	88 % (351/400)	12 % (49/400)	0.035
SenseLab	33 % (25/75) [14, 56]	63 % (41/65) [40, 76]	96 % (134/140)	81 % (97/120)	92 % (368/400)	8 % (32/400)	0.033

Table 1: After the rule changes conditionally (set C, change at episode 30 of 100; GPT-5.5; four cells per system: `support` and `ci-fix` × seeds 11 / 23). First-attempt success on every post-change episode. *Moved half*: changed-class instances whose rule changed; *kept half*: changed-class instances whose rule did not. Brackets: cell-level bootstrap 95% interval. Development seeds; SenseLab on the `post-canary-ni` engine cut.

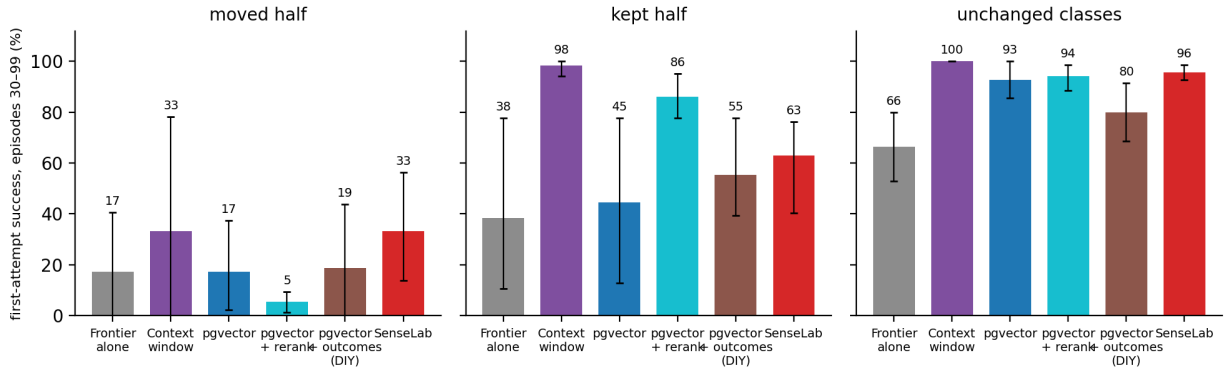


Figure 1: First-attempt success after the rule changes conditionally (episodes 30–99, four cells, GPT-5.5): the moved half, the kept half and the unchanged classes, one bar per system, with cell-level bootstrap 95 % intervals. Same numbers as Table 1.

Table 1 is the engine on its current cut, `post-canary-ni` (findings 8–11 live). On the moved half every retrieval memory is at or below the frontier model alone. A note that says "do X for class C" is retrieved as confidently after the change as before, and pgvector with reranking, the best retriever, is the worst system on the moved half at 5 % because it retrieves the stale note most reliably. SenseLab is the only system that recovers the moved half on both families while holding the unchanged classes at 96 %: 33 % [14, 56] against 5–19 % for the frontier model and the three vector stores, winning every paired cell. The context window ties it at 33 %, by beating it on `ci-fix` (78 against 56; the whole history fits in 24k tokens and the model can see the change

happen) and doing nothing on `support` (0/43, SenseLab 7/43), at 1.5× the all-in cost and 3.7× the tokens. SenseLab pays on the kept half: 63 % against 86 % and 98 % for pgvector + rerank and the context window, the two systems that keep applying the old rule most reliably. That loss is the mechanism at work: the entry is marked contested and the agent explores where the old action was still right. It is also the number the repair loop exists to remove, since a written conditional rule would recover the moved half without giving up the kept one. Whether it does yet is Table 4.

pair	support s11	support s23	ci-fix s11	ci-fix s23	mean	cells won	p	p (Holm)
SenseLab − Frontier alone	+26	+5	+11	+23	+16	4/4	0.125	0.625
SenseLab − Context window	+26	+5	-26	-15	-3	2/4	0.875	0.875
SenseLab − pgvector	+22	+5	+11	+31	+17	4/4	0.125	0.625
SenseLab − pgvector + rerank	+22	+5	+47	+46	+30	4/4	0.125	0.625
SenseLab − pgvector + outcomes (DIY)	+26	+5	+5	+23	+15	4/4	0.125	0.625

Table 2: SenseLab minus each system on the moved half, per cell, in percentage points, paired on (family, seed).

system	support: moved	support: kept	support: unchanged	ci-fix: moved	ci-fix: kept	ci-fix: unchanged
Frontier alone	0 % (0/43)	78 % (21/27)	53 % (37/70)	41 % (13/32)	11 % (4/38)	80 % (56/70)
Context window	0 % (0/43)	96 % (26/27)	100 % (70/70)	78 % (25/32)	100 % (38/38)	100 % (70/70)
pgvector	2 % (1/43)	78 % (21/27)	86 % (60/70)	38 % (12/32)	21 % (8/38)	100 % (70/70)
pgvector + rerank	2 % (1/43)	78 % (21/27)	90 % (63/70)	9 % (3/32)	92 % (35/38)	99 % (69/70)
pgvector + outcomes (DIY)	0 % (0/43)	78 % (21/27)	69 % (48/70)	44 % (14/32)	39 % (15/38)	91 % (64/70)
SenseLab	16 % (7/43)	44 % (12/27)	96 % (67/70)	56 % (18/32)	76 % (29/38)	96 % (67/70)

Table 3: By family: first-attempt success on the moved half, the kept half and the unchanged classes.

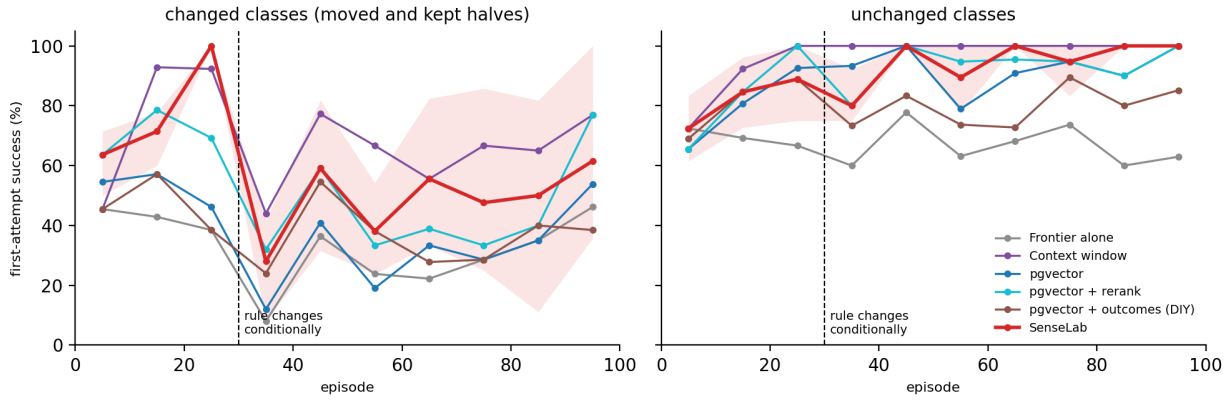


Figure 2: First-attempt success by 10-episode block across the change at episode 30, pooled over the four cells: the classes whose rule changes (left; before the change every system is at 80–90 %, after it the rule holds for the kept half only) and the unchanged classes (right). SenseLab is drawn heavy with its cell-level bootstrap band; the other systems' bands are of the same width and are omitted for legibility. The context window recovers most on `ci-fix` because the whole history fits in its prompt (Table 3).

4.2 Attribution: what level 1 adds to level 0, so far

system	moved half	kept half	unchanged classes	pre-change	episode success	escalated	infra err	$/ep
SenseLab	33 % (25/75) [14, 56]	63 % (41/65) [40, 76]	96 % (134/140)	81 % (97/120)	92 % (368/400)	8 % (32/400)	0	0.033
SenseLab − canary	39 % (29/75) [14, 66]	62 % (40/65) [42, 74]	99 % (139/140)	79 % (95/120)	91 % (365/400)	9 % (35/400)	0	0.033
SenseLab − repair loop	51 % (38/75) [33, 73]	69 % (45/65) [58, 76]	96 % (135/140)	81 % (97/120)	92 % (369/400)	8 % (31/400)	0	0.032

Table 4: Attribution. SenseLab against its own ablations on the same protocol: without the canary (policy `auto_after_replay`, same run) and without the repair loop (policy `review`: proposals never approved; outcome learning alone). This is the only table in which the ablations appear.

On these seeds level 1 does not yet add to level 0. Without the repair loop (policy `review`: the loop proposes, nothing ships, the memory learns from outcomes alone) the moved half is 51 % and the kept half 69 %; with it, 33 % and 63 %; without the canary but with the replay gate, 39 % and 62 %. The intervals overlap throughout. The lifecycle table shows why, and it is not that the loop was idle.

arm	proposed	regime / judge	T1 pass / inc. / fail	T2 pass / fail / open	canaries run / undecided	shipped
SenseLab, pre-seam4	9	5 / 4	7 / 2 / 0	2 / 2 / 0	3 / 3	0
SenseLab − canary, pre-seam4	8	4 / 4	4 / 3 / 1	1 / 2 / 0	0 / 0	1
SenseLab, post-seam5	16	12 / 4	5 / 10 / 1	6 / 3 / 7	4 / 3	0
SenseLab − canary, post-seam5	17	13 / 4	12 / 4 / 1	9 / 7 / 1	0 / 0	9
SenseLab, post-seam5b	24	19 / 5	14 / 9 / 1	15 / 8 / 1	6 / 4	0
SenseLab − canary, post-seam5b	16	13 / 3	10 / 5 / 1	11 / 4 / 0	0 / 0	11
SenseLab, post-clean-success	16	13 / 3	9 / 7 / 0	3 / 13 / 0	3 / 3	0
SenseLab − canary, post-clean-success	17	13 / 4	9 / 4 / 4	2 / 12 / 3	0 / 0	2
SenseLab, post-canary-ni	13	8 / 5	4 / 8 / 0	4 / 7 / 1	3 / 3	0
SenseLab − canary, post-canary-ni	14	10 / 4	4 / 9 / 1	0 / 13 / 0	0 / 0	0

Table 5: Fix lifecycle by run and arm. Tier 1: stand-in replay on the fix branch; Tier 2: the customer's replay of the case set; a canary grades live exposure in the fix's situation. *open*: Tier 2 replays still awaited when the cell ended. No canary reached a decision inside a 100-episode cell; no fix was reverted; 96–100% of proposals were drafted by the repair agent.

4.3 What level 1 did: the gating record

The repair loop did not move the outcome endpoints on these seeds. What it did is refuse. Every fix it drafted went through the proof set and, if that passed, the live canary, and on the current cut nothing shipped: the proof rejected the drafts that restated or flipped the rule wholesale, and the canary held the rest because the failure they were written for was no longer live. A loop that ships only what it can prove, and only against a failure it can still observe, is slower than one that ships what looks right. The record below is what that slowness bought and cost; the confirmatory seeds re-measure it on the same protocol (§5).

What the loop did. On the current cut the loop proposed 13 fixes in the four SenseLab cells, 8 from regime shifts. Four passed Tier 1 (8 inconclusive), 4 passed Tier 2 and 7 failed it (one open), and none shipped, because no canary decided before the cell ended. Three canaries were running at episode 100. Two had 2/8 and 4/8 canary-arm runs graded against the floor. The third was fully graded (18 canary-arm and 31 control-arm runs) and still had no verdict: at 6 % against 10 % recurrence the one-sided bound on the difference was 13 points against a margin of 10. With a 10 % control recurrence the bound falls under the margin at about 15 graded runs per arm if the canary never recurs, about 30 if it recurs at 6 %, and about 50 if it recurs as often as control; a 100-episode cell supplied 18 and 31 to its best canary. Two cuts earlier the same protocol passed 15 of 24 at

Tier 2 and the no-canary ablation shipped 11 of 16; on the previous cut the ablation shipped 2 of 17; on this cut it shipped none of 14 (Tier 2: 0 passed, 13 failed). With 27–43 episodes per half per family, none of the outcome differences between the arms is a difference.

What the proof rejects. Three cuts earlier, hand-coding the fix text against the true change, 20 of the 24 regime fixes that passed their proof set encoded an unconditional flip ("always Y") where the change was conditional ("Y when a, else X"); on `ci-fix`, 19 of 19. The proof graded a replay on the outcome it reported, or on a judge whose rubric passes a recovered run, and an agent with a retry budget recovers, so a wholesale flip passed every case it could. Since the `post-clean-success` cut a proof case passes only on a clean success, whatever grades it. On that cut Tier 2 failed 19 of the 23 regime fixes that reached it; on the current cut, with the repair agent also handed the situation's passing runs, it failed 15 of 17: 4 on a kept case (a flip broke the half the old rule still holds for), 10 on a moved case only (the old rule in better words did not repair the half that moved), 1 on a pre-change case. The two that passed, both in canary and neither shipped, are a wholesale flip on `ci-fix` (the opposite of the true condition) and a retry ladder on `support` that branches on what has already failed, not on the app version the change turned on. Their replays succeeded cleanly anyway, because the replaying agent read the rule and did not follow it into the wrong action. A proof pass is therefore necessary and cannot by itself certify a conditional when the agent's compliance with an entry is partial; that is what the live canary is for.

What the drafts say now. Amendment (v) changed the draft: the repair agent is shown the situation's passing runs and asked for the condition that separates them from the failing ones. P11(a′) asks whether at least half of the regime drafts whose proof set holds a kept case state a condition. Coded by the author against written instructions (amendment (y); every text, the coding and the key are in the package), 21 of the 37 such drafts on this cut (57 %) state an input-dependent branch: met. The split by class is the finding. On `ci-fix` 11 of 14 drafts are conditional and every one names the true variable (whether the PR touches `legacy_sdk/`). On `support` 9 of 23 are conditional, and they branch on what is visible in the ticket (platform or plan), not on what the world changed on: the plan is named by 2 of 7 export-timeout drafts, the issuer country by 0 of 8 card-declined drafts, the app version by 0 of 6 notification drafts. The drafter writes conditions where the evidence puts the variable in front of it, as file paths in a diff do, and writes the salient attribute where it does not. The confirmatory fleet-1 drafts (39, same sheet, same key) read the same way: 27 of 39 (69 %) state a condition; on `ci-fix` 20 of 28, 17 of them on the PR's files; on `support` 7 of 11, none on the true variable.

4.4 What the replay gate alone let through (exploratory)

No canary decided inside a cell, so the canary's value cannot be read from Table 4. What can be read, after the fact and outside the preregistration, is what happened to traffic once the no-canary ablation shipped: 21 ships over nine cells in three dev runs, every one a regime fix that had passed Tier 1 and Tier 2. Table 6 takes each such cell from its first ship to its end and compares the two halves of the changed class with the SenseLab cell of the same family, seed and cut over the same episodes, which shipped nothing.

population	−canary before	−canary after	SenseLab before	SenseLab after	DiD
kept	13/46 (28%)	29/99 (29%)	11/46 (24%)	43/99 (43%)	-18%
moved	50/59 (85%)	68/111 (61%)	46/59 (78%)	52/111 (47%)	+8%

Table 6: Exploratory, not preregistered: traffic after the no-canary ablation shipped, against the SenseLab cell of the same family, seed and cut over the same episodes (nine cells with a ship; window from the first ship to the end of the cell). *Recurrence* is the canary's reading of a session: not a first-attempt success. DiD = $(\text{after} - \text{before})_{-\text{canary}} - (\text{after} - \text{before})_{\text{SenseLab}}$; negative means the ships were associated with less recurrence than shipping nothing.

The ships did not break the kept half. Kept-half recurrence stayed flat in the ablation (28 % to 29 %) while it rose in the SenseLab cell over the same episodes (24 % to 43 %); on the moved half the ships went with eight points less recovery. Read with the hand-coding above, this is one mechanism: a correction that restates the old rule in better words protects the half the old rule still holds for and slows the half it does not. It also shows what level 0 does alone when a validated entry starts failing conditionally: it demotes the entry for everyone, including the half it was still right for. The kept-half loss in Table 1 is that demotion, and the repair loop is the only level that can write the condition back. These corrections did not damage the situation on net, so a gate that compares arms would have found little to stop; the canary's value has to be read where canaries decide and against corrections that do net damage.

Where canaries could decide. We ran the E2 shape on the development seeds (`rsi-e2b-c5`, cut `post-clean-success`): ten agents sharing one store, 200 episodes with the change at 50, three arms (SenseLab, −canary, −replay), 2,400 episodes. On `ci-fix` every arm recovers both halves by episode 130 (moved 93–100 %, kept 94–96 % over episodes 130–199); on `support` none does (moved 26–32 %, kept 51–63 %). The arms are indistinguishable at this sample. Twenty canaries started and none decided: of 16–30 exposed runs per arm, 0–8 could be graded, because grading was by canonical situation and the agent's declarations had been folded to other canonicals than the fix's (finding 11). The rerun on the corrected cut (`rsi-e2b-c6`, `post-canary-ni`; 2,400 episodes) read P12–P14. The outcome picture did not move (`ci-fix` recovered by episode 130 in every arm; `support` moved half 38 % for SenseLab against 35 and 22 for the ablations). Nine canaries started, and Figure 3 is their time course. The share of exposed runs the canary could grade rose from 0–27 % to 12–55 %, four of nine reached the floor of eight graded runs on both arms (P12 not met at 4 of 9, not falsified), and none decided (P13 falsified). Three of the four at the floor had no recurrence in either arm; the fourth had 0 % against 10 % at 9 and 10 graded, a bound of 14 points against the margin of 10. We read this at the time as the fleet recovering before the gate could measure. Finding 12, made the next morning from the loop's own read record, showed that no canary-arm run in any of these nine canaries had read the fix's entry; on the branch the arm had in fact lost the key. The canaries compared two untreated arms. The confirmatory fleet-10 cells (§5) are the first run in which a canary arm holds the correction it is graded on.

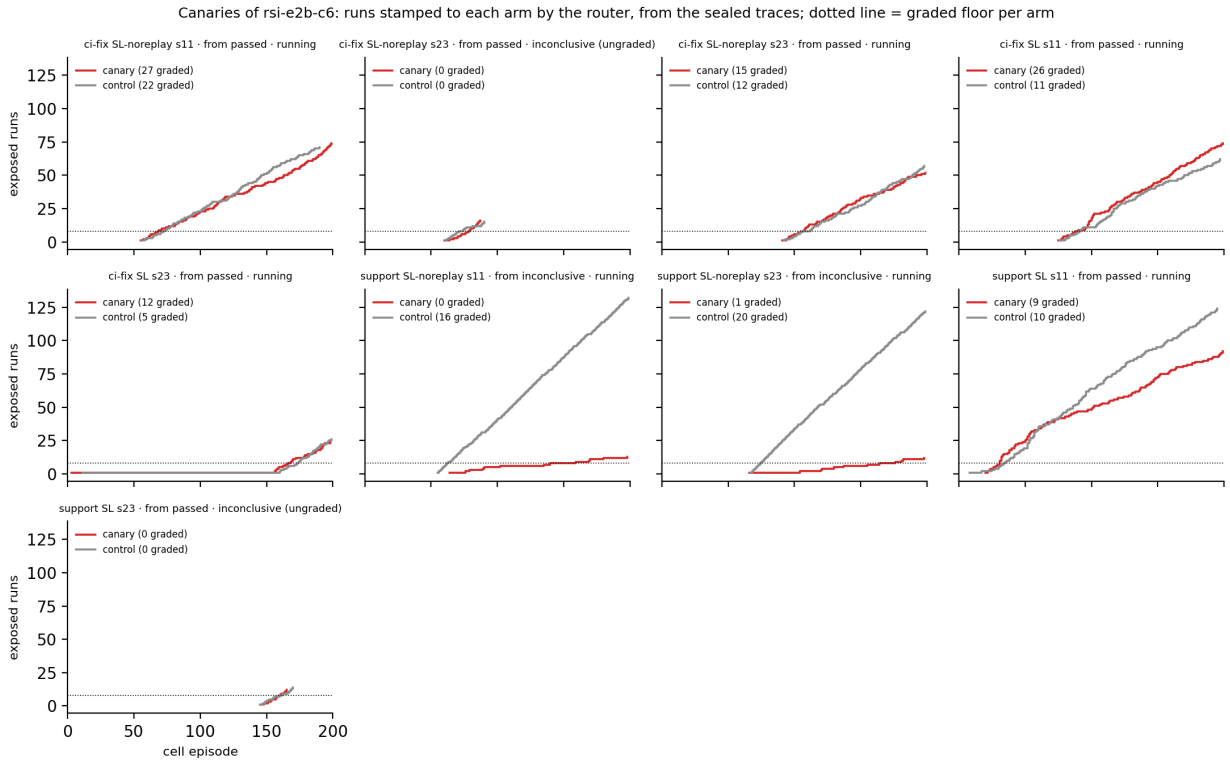


Figure 3: The nine canaries of `rsi-e2b-c6` (fleet 10, 200 episodes, change at 50), one panel per fix: runs the router stamped to the canary arm (red) and the control arm (grey) by cell episode; dotted line, the graded floor of 8 per arm; the legend gives how many stamped runs the canary could grade. Four reached the floor on both arms; none decided. Finding 12 later showed that no canary-arm run had read the fix.

4.5 SenseLab by engine cut

engine cut	moved half	kept half	unchanged	escalated	fixes proposed	Tier 2 passed	shipped
pre-seam4 (‘rsi-c1-rsi‘)	36 % (27/75)	65 % (42/65)	96 % (134/139)	6 % (22/399)	9	2	0
post-seam5 (‘rsi-c2‘)	47 % (35/75)	60 % (39/65)	99 % (138/140)	6 % (24/400)	16	6	0
post-seam5b (‘rsi-c3‘)	43 % (32/75)	60 % (39/65)	99 % (138/140)	8 % (30/400)	24	15	0
post-clean-success (‘rsi-c5‘)	34 % (25/74)	63 % (41/65)	96 % (132/137)	8 % (33/396)	16	3	0
post-canary-ni (‘rsi-c6‘)	33 % (25/75)	63 % (41/65)	96 % (134/140)	8 % (32/400)	13	4	0

Table 7: SenseLab on successive cuts of the engine, same protocol and seeds. Fix counts are over the four SenseLab cells of each run. `pre-seam4`: amfs-internal #787/#788; `post-seam5`: #793, #794; `post-seam5b`: #795, #796; `post-clean-success`: #797, #798; `post-canary-ni`: #799 (§6).

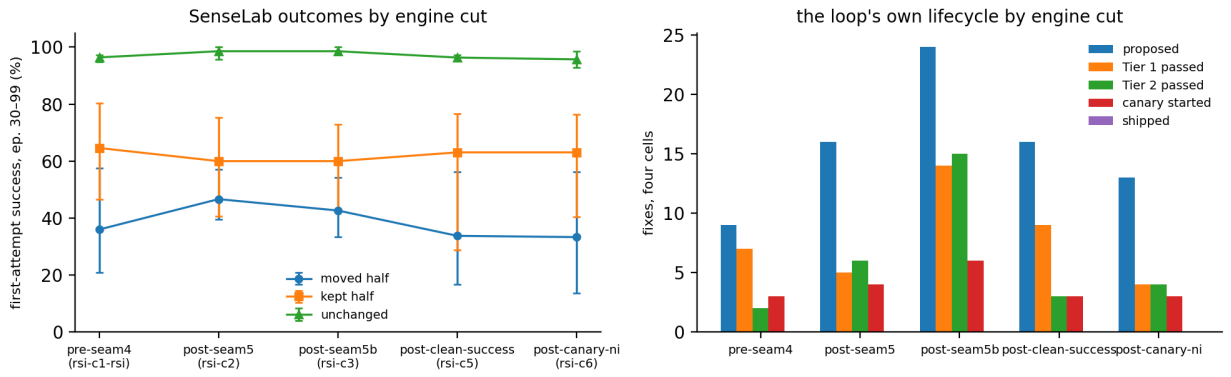


Figure 4: SenseLab on successive cuts of the engine, same protocol and seeds. Left: outcome endpoints, which move within their intervals. Right: the loop's lifecycle counts over the four cells. Nothing shipped on any cut, since no canary decided inside a 100-episode cell.

Table 7 is the same protocol on successive cuts and is §6 in numbers. The loop went from 9 proposals and 2 Tier 2 passes on the first cut to 24 and 15 on the third as its evidence path was repaired, then to 16 and 3, and 13 and 4, as the proof learned to reject a recovered success. The outcome endpoints moved within noise throughout.

4.6 Two probes on the earlier change set

When the team grows. On the unconditional change (set A, change at 20), with a fleet of six agents sharing one store: in the *pioneer* protocol one agent works alone for 60 episodes and five join; in the *newcomer* protocol five work and a sixth joins at 60. A joiner on SenseLab takes the post-change action on its first exposure to a changed class in 36/36 (pioneer) and 10/10 (newcomer) cases with zero stale picks. A joiner on pgvector + rerank does so in 31 % and 40 % of cases, with 17 % and 0 % stale picks and 16–20 % escalations. The same notes are in both stores; what the joiner inherits is the outcome record.

arm	veterans ep50-59 1st	veterans changed post 1st	joiner 1st-exp all	1st-exp changed	1st-exp unchanged	stale-pick (changed 1st-exp)	escalated
pgvector + rerank	0.75	0.28	0.68 [0.52,0.83] (n=90)	0.31 [0.15,0.50] (n=36)	0.93 [0.78,1.00] (n=54)	0.17	0.20
SenseLab	1.00	0.76	1.00 [1.00,1.00] (n=90)	1.00 [1.00,1.00] (n=36)	1.00 [1.00,1.00] (n=54)	0.00	0.00

Table 8: When the team grows (set A, change at 20, fleet of six sharing one store; *pioneer*: one agent works alone to episode 60, five join). First-attempt success of the joiners on their first exposure to each class after joining, with the veterans' rate before they joined; *stale-pick*: the joiner applied the pre-change action.

A student trained on the record (E7 at rung 0). Two Managed Models (`senselab-s1`) were trained on the dev-seed `support` traces of SenseLab and of the ablation without the repair loop (200 traces each), and evaluated on held-out seeds 101/113/127 (1,500 episodes). Both were served under a relaxed promotion gate; the default gate would have rejected both on exact agreement with their teacher (0.00 against a floor of 0.45), as amendment (o) requires us to state. Alone, neither student takes the new action first (9 % and 0 % on the moved half). With SenseLab's outcome loop as its memory each does (33 % and 30 %), at 1/23 of the frontier model's cost per episode, and beats GPT-5.5 with the same memory (22 %) on the moved half at this rung. That is the contamination the evaluation ladder exists to expose: rung 0 uses the training classes. The SenseLab-trained student against the ablation-trained one is $-7/0/+18$ points across the three seeds, which the preregistration calls "small and reported as such". Nothing is claimed from rung 0; rungs 1–3, on

classes and wordings the students never saw, need eval-only content that is registered but not yet written.

5 Confirmatory results

Seeds 37, 41, 59 and 73 ran once, on both agent models, after the analysis code was tagged `analysis-freeze-rsi`. Every table in this section is filled by that frozen code from the confirmatory read. The endpoints, the paired test (exact two-sided sign-flip over the sixteen cells, Holm across the five systems on the primary endpoint, Holm within each secondary endpoint) and the tables are those of amendments (w) and (x). The prose was written after the read and states, for each preregistered prediction, whether it was met.

How the fleet-1 read was made. The four seeds ran as four per-seed runs (`rsi-conf-f1-s{37,41,59,73}`, 3,200 episodes each, 01:42–08:19Z on 2026-09-30, all before the `post-canary-visible` cut of finding 12, so every fleet-1 canary arm is untreated by construction). The frozen reader resolves one run per system, so the four were concatenated file-by-file into `rsi-conf-f1` before the read. Two wiring defects in the frozen reader (paired tables wired to the development run names; the lifecycle table iterating the development cut list) were corrected before any confirmatory number was read from a complete table. Neither touches a metric, endpoint, test statistic or population rule, and the development read is byte-identical before and after; the correction is commit `e04f78cf`, the one post-freeze change to analysis code. One engine cut fell inside the window, `post-canary-zero` (OSS #454, amfs-internal #800/#801, 03:40:19Z), with 4,867 of the 12,791 episodes before it. The moved-half difference between SenseLab and every other system has the same sign on the pre-cut episodes alone (+28/+11/+6/+18/+23 pp, on 75 SenseLab episodes) as on the full read (+19/+4/+13/+9/+13), so the cut did not make the result.

system	moved half	kept half	unchanged classes	pre-change	episode success	escalated	$/ep
Frontier alone	25 % (72/284) [13, 39]	27 % (75/276) [16, 39]	77 % (432/560)	65 % (312/480)	85 % (1362/1600)	15 % (238/1600)	0.014
Context window	40 % (115/284) [27, 54]	92 % (253/276) [84, 97]	97 % (542/560)	86 % (415/480)	92 % (1476/1600)	8 % (124/1600)	0.130
pgvector	32 % (91/284) [20, 45]	66 % (183/276) [59, 74]	96 % (539/560)	82 % (392/480)	92 % (1469/1600)	8 % (131/1600)	0.024
pgvector + rerank	36 % (101/284) [23, 48]	66 % (183/276) [58, 75]	91 % (511/560)	77 % (371/480)	90 % (1435/1600)	10 % (165/1600)	0.027
pgvector + outcomes (DIY)	32 % (90/284) [20, 44]	58 % (161/276) [48, 69]	84 % (470/560)	76 % (364/480)	88 % (1411/1600)	12 % (189/1600)	0.032
SenseLab	45 % (127/283) [39, 51]	69 % (191/276) [62, 76]	100 % (556/557)	85 % (406/480)	95 % (1516/1596)	5 % (80/1596)	0.033

Table 9: Confirmatory: after the rule changes conditionally. Sixteen cells per system (`support`, `ci-fix` × four seeds × two models); first-attempt success on every post-change episode; brackets: cell-level bootstrap 95% interval. Confirmatory seeds 37 / 41 / 59 / 73, both agent models (GPT-5.5, Claude Opus 5), the two set-C families, run once on the `post-canary-ni` cut after `analysis-freeze-rsi` (amendment (w)).

pair	support gpt-5.5	support claude-opus-5	ci-fix gpt-5.5	ci-fix claude-opus-5	mean	cells won	p	p (Holm)
SenseLab − Frontier alone	+28	+56	-9	-1	+19	11/16	0.024	0.121
SenseLab − Context window	+17	+36	-17	-19	+4	8/16	0.544	0.544
SenseLab − pgvector	+23	+16	+22	-11	+13	11/16	0.040	0.159
SenseLab − pgvector + rerank	+24	+7	+3	+4	+9	9/16	0.119	0.239
SenseLab − pgvector + outcomes (DIY)	+23	+34	-6	+1	+13	10/16	0.040	0.159

Table 10: Confirmatory primary endpoint: SenseLab minus each system on the moved half, percentage points, paired on (family, seed, model); per family × model means, the mean over the sixteen cells, cells won, the exact two-sided sign-flip p over the cells and its Holm adjustment across the five systems (amendment (x)).

Primary endpoint: the moved half. On the half of the changed classes whose rule moved, SenseLab's first-attempt success after the change is 45 % (127/283, cell-level interval [39, 51]) against 25–40 % for the five baselines. Paired on (family, seed, model) over the sixteen cells, the mean delta is positive against every baseline: +19 pp over Frontier alone, +4 over the context window, +13 over pgvector, +9 over pgvector with a reranker, +13 over the DIY outcome store. SenseLab wins 8 to 11 of the 16 cells in each pairing. The exact sign-flip p is 0.024 against Frontier alone, 0.040 against pgvector and the DIY store, 0.119 against the reranker and 0.544 against the context window; none survives Holm across the five systems at 0.05 (0.121, 0.544, 0.159, 0.239, 0.159). The primary prediction, SenseLab ahead of every baseline on the moved half, is met in sign and in cells won, and not at the preregistered significance. Sixteen cells is the power the reduced confirmatory scope of amendment (w) bought. The deltas are of the size the development seeds showed; read them as estimates with their intervals.

pair	support gpt-5.5	support claude-opus-5	ci-fix gpt-5.5	ci-fix claude-opus-5	mean	cells won	p	p (Holm)
SenseLab − Frontier alone	+1	+23	+53	+75	+38	14/16	0.002	0.007
SenseLab − Context window	-36	-12	-26	-21	-24	1/16	0.000	0.001
SenseLab − pgvector	+2	+8	-13	+9	+1	9/16	0.834	1.000
SenseLab − pgvector + rerank	-2	+1	+1	+3	+1	9/16	0.882	1.000
SenseLab − pgvector + outcomes (DIY)	-17	+3	+23	+25	+8	10/16	0.236	0.708

Table 11: Confirmatory secondary endpoint, the kept half: SenseLab minus each system, paired as in Table 10; Holm within this endpoint.

pair	support gpt-5.5	support claude-opus-5	ci-fix gpt-5.5	ci-fix claude-opus-5	mean	cells won	p	p (Holm)
SenseLab − Frontier alone	+37	+35	+19	+0	+23	12/16	0.000	0.002
SenseLab − Context window	+4	+9	-1	+0	+3	4/16	0.125	0.125
SenseLab − pgvector	+4	+4	+0	+6	+4	6/16	0.031	0.062
SenseLab − pgvector + rerank	+21	+14	-1	+0	+9	8/16	0.012	0.035
SenseLab − pgvector + outcomes (DIY)	+26	+25	+12	+0	+16	11/16	0.001	0.004

Table 12: Confirmatory secondary endpoint, the unchanged classes: SenseLab minus each system, paired as in Table 10; Holm within this endpoint.

Secondary endpoints: collateral damage. On the kept half, the changed classes whose rule did not move, SenseLab is at 69 % (191/276): above Frontier alone by 38 pp (14/16 cells, Holm 0.007), level with the three pgvector arms (deltas +1, +1, +8; none distinguishable), and below the context window by 24 pp (1/16 cells, Holm 0.001). The context window carries every prior note and outcome of the cell in the prompt. On a rule that did not change, replaying the whole record is the right policy and it wins, at $0.130 per episode against SenseLab's $0.042 all in (Frontier alone $0.014, the pgvector arms $0.024–0.032; SenseLab's figure includes the loop's judges, replays and drafts). This is the cost of the moved-half gain on this protocol: SenseLab's memory is selective and outcome-weighted, and on the kept half selection loses to exhaustive recall. On the unchanged classes SenseLab is at 100 % (556/557), above Frontier alone (+23, Holm 0.002), the reranker (+9, 0.035) and the DIY store (+16, 0.004), and not distinguishable from the context window or plain pgvector. No baseline is above it.

system	support: moved	support: kept	support: unchanged	ci-fix: moved	ci-fix: kept	ci-fix: unchanged
Frontier alone	0 % (0/150)	52 % (68/130)	64 % (179/280)	54 % (72/134)	5 % (7/146)	90 % (253/280)
Context window	17 % (25/150)	92 % (119/130)	94 % (262/280)	67 % (90/134)	92 % (134/146)	100 % (280/280)
pgvector	22 % (33/150)	62 % (80/130)	96 % (269/280)	43 % (58/134)	71 % (103/146)	96 % (270/280)
pgvector + rerank	27 % (40/150)	66 % (86/130)	82 % (231/280)	46 % (61/134)	66 % (97/146)	100 % (280/280)
pgvector + outcomes (DIY)	14 % (21/150)	74 % (96/130)	74 % (208/280)	51 % (69/134)	45 % (65/146)	94 % (262/280)
SenseLab	42 % (62/149)	69 % (90/130)	100 % (277/277)	49 % (65/134)	69 % (101/146)	100 % (279/280)

Table 13: Confirmatory, by family: first-attempt success on the moved half, the kept half and the unchanged classes.

system	gpt-5.5: moved	gpt-5.5: kept	gpt-5.5: unchanged	claude-opus-5: moved	claude-opus-5: kept	claude-opus-5: unchanged
Frontier alone	25 % (36/142)	28 % (39/138)	72 % (201/280)	25 % (36/142)	26 % (36/138)	82 % (231/280)
Context window	36 % (51/142)	88 % (122/138)	98 % (275/280)	45 % (64/142)	95 % (131/138)	95 % (267/280)
pgvector	13 % (18/142)	63 % (87/138)	98 % (274/280)	51 % (73/142)	70 % (96/138)	95 % (265/280)
pgvector + rerank	22 % (31/142)	57 % (79/138)	90 % (251/280)	49 % (70/142)	75 % (104/138)	93 % (260/280)
pgvector + outcomes (DIY)	27 % (38/142)	54 % (74/138)	80 % (225/280)	37 % (52/142)	63 % (87/138)	88 % (245/280)
SenseLab	36 % (51/142)	60 % (83/138)	100 % (279/280)	54 % (76/141)	78 % (108/138)	100 % (277/277)

Table 14: Confirmatory, by agent model: first-attempt success on the moved half, the kept half and the unchanged classes.

By family and by model. The moved-half gain is a `support` result. There SenseLab is at 42 % (62/149) against 0–27 % for every baseline, with Frontier alone at 0/150. On `ci-fix` it is at 49 % (65/134) against 43–67 %, behind the context window (67 %). By model, SenseLab is level with the context window on GPT-5.5 (36 % both) and ahead on Claude Opus 5 (54 % against 45 %, the next best); the pgvector arms swap order across the two models (13–27 % on GPT-5.5, 37–51 % on Opus). Kept-half and unchanged-class rates are within a few points across models for every system; the model effect is on the moved half.

system	moved half	kept half	unchanged classes	pre-change	episode success	escalated	infra err	$/ep
SenseLab	45 % (127/283) [39, 51]	69 % (191/276) [62, 76]	100 % (556/557)	85 % (406/480)	95 % (1516/1596)	5 % (80/1596)	4	0.033
SenseLab − canary	44 % (125/283) [36, 54]	71 % (197/276) [66, 76]	99 % (548/556)	86 % (412/480)	94 % (1506/1595)	6 % (89/1595)	5	0.034
SenseLab − repair loop	43 % (122/284) [33, 52]	68 % (187/276) [59, 76]	99 % (557/560)	85 % (410/480)	93 % (1494/1600)	7 % (106/1600)	0	0.033

Table 15: Confirmatory attribution: SenseLab against its own ablations on the same sixteen cells. The only confirmatory table in which the ablations appear.

arm	proposed	regime / judge	T1 pass / inc. / fail	T2 pass / fail / open	canaries run / undecided	shipped
SenseLab, confirmatory	32	23 / 9	16 / 13 / 3	6 / 22 / 2	5 / 3	2
SenseLab − canary, confirmatory	44	32 / 12	25 / 14 / 4	10 / 32 / 0	0 / 0	10

Table 16: Confirmatory fix lifecycle by arm over the sixteen fleet-1 cells (the fleet-10 cells are read in §5).

Attribution and the loop's lifecycle. Inside a 100-episode cell at fleet size 1, the two ablations are indistinguishable from the full system: without the canary 44 / 71 / 99 % (moved / kept / unchanged), without the repair loop 43 / 68 / 99, against 45 / 69 / 100. This is the same reading as on the development seeds, and the reason the long-horizon protocol exists. The loop was active: 32 fixes proposed in the sixteen SenseLab cells (23 from a regime shift, 9 from the judges), 16 passing the stand-in replay, 6 the customer replay, 5 canaries opened, 3 undecided when their cell ended, 2 shipped, 0 reverted. With the canary removed the same loop shipped 10 of 44. The canary gate held back four fixes in five that the replay would have shipped, in cells where none could have been graded to a decision.

The long horizon: fleet 10, 200 episodes. The E2b protocol on the four confirmatory seeds runs SenseLab alone on GPT-5.5 (amendment (y); the gate ablations and the Opus cells are not run, since two development runs had already read them the same way). It supplies the learning curve on held-out seeds, descriptively, and the canary readout: P12 (canaries reach the graded floor on both arms), P13 (canaries decide), P14 (no pass harms the kept class by more than the margin), P13′/P13″/P15′/P16 of amendments (z) and (z′) (the three classes; canary-arm reads of the fix), and P11(a′). The protocol ran twice. The first four cells (`rsi-conf-e2b-s{37,41,59,73}`, on the `post-canary-visible` cut) are the read of finding 13: the arm read its fix for the first time in the project, and the loop's own replays discredited five of the six corrections within minutes. They are reported in §6 as measured. The confirmatory read is the rerun on the `post-replay-evidence` cut (amendment (aa); `rsi-conf-e2b-r2-s{37,41,59,73}`, 12:59–16:08Z on 2026-09-30, 400/400 episodes each).

Over the eight cells the loop proposed 48 fixes (39 from a regime shift, 9 from the judges). The stand-in replay passed 23 and was inconclusive on 23; the customer's replay passed 6 and failed 38; 6 canaries opened, in six different cells; 0 shipped, 0 reverted. The canary arm now reads its fix: in the sessions the canary graded, the fix's own key was a version-exact read in 57 of 76 (75 %), and the control arm read it in 0 of 54 (P16 met; the engine's per-arm `read` count and the version-exact join agree within three sessions on every canary). Reads per canary arm at close were 2, 5, 8, 12, 14 and 18, median 10 against the floor of 8 (P13″ met; finding 12's run had 0, 0, 1, 1, 1 and 17). Four of six canaries reached the graded floor on both arms (P12 met), with a graded share of routed sessions of 0.23–0.36 on five and 1.0 on one, so P12's second clause (≥ 0.8) is not met: exposure counts the arm's routing, grading counts the situation. One canary decided within its cell: `8a908156` (`support`, seed 37, "Android free-plan push notifications stopped") failed at 10/10 graded sessions on the zero-count bound, with the fix read in 5 of the arm's 10 graded sessions, below the read floor. By the class rule of amendment (z″) that is a verdict on a partly treated arm, so P13′ is not met and not falsified; the falsifier (0 verdicts in ≥ 6 canaries) is not met by the letter and is by the class rule, and we report both. Three others closed with both arms above the floor (20/8, 14/10, 17/14) and no decision; at these counts the non-inferiority test cannot close a 10-point margin either way. That is the gate's power at this horizon, not a defect. The treated kept-class comparisons of P15′ are all under its 8-session floor (1.00 (6) against 0.88 (17), 1.00 (6) against 0.77 (22), 0.33 (6) against 0.40 (25), 0.00 (3) against 0.70 (10)). P14 and P15 have no population, since nothing passed.

Live evidence reached the corrections this time. Five of the six fixes wrote a correction, and every one was charged by canary-arm sessions that had read it: at close 14 wins and 0 failures (0.82), 12 and 2 (0.65), 8 and 2 (0.47, discredited at the second failure and pinned all the same), and, for the two rewrites of the agent's own entries born at 0.37 and 0.30 under the 0.8× rule, 7 and 26 and 1 and 3, both discredited within two minutes of their first read. Whether the discredit threshold should apply to a correction under test is an engine question this version leaves open. No replay wrote an outcome row or a contrast lesson after the cut: 209 replays sealed in the window, 0 of 1,600 outcome rows from them.

The long horizon (descriptive, per amendment (x); pooled over the eight cells with a cell-level bootstrap): moved-half first-attempt success 37 % (40/109) in episodes 50–99, 55 % (60/110) in 100–149 and 75 % (67/89) in 150–199, with 88 % (15/17) in the last block; kept half 66, 76 and 72 %; unchanged classes 100 % in every post-change block (594/595). The first run of the same cells, on the cut before, read 32, 59 and 67 % on the moved half. At 200 episodes with ten agents, the system that could not be separated from its level-0 ablation inside a 100-episode cell reaches three quarters of the moved half, and the canary, now treating the arm it grades, holds every one of the six fixes it opened: five for want of power, one on a failure it saw.

Exploratory: the canary on the traffic its rule is sized for (amendment (ab)). This block is exploratory: one run on the development seeds, declared in amendment (ab) before its first episode and after every confirmatory cell had been read. It enters no confirmatory table, no ledger row and no confirmatory number above. The confirmatory canaries had 14–20 graded runs per arm; the engine's pass bound at ∼10 % recurrence is 0.175 at 20, 0.118 at 40, 0.105 at 50 and 0.095 at 60 against a margin of 0.10, so the question was what the gate does with the traffic its rule is sized for. The design is the E2b cell with `--episodes 400` (canary timeout and window 400) and `--change-focus 1.0`, so every post-change episode comes from the changed classes and a fix's situation receives about twice the traffic. The unchanged classes are not measured after episode 50, and no safety claim is read from this run. SenseLab only, GPT-5.5, set C, fleet 10, `support` and `ci-fix`, seeds 11 and 23: four cells, 1,600 episodes, on the `post-replay-evidence` production, 21:37Z 2026-09-30 to 03:50Z 2026-10-01, read with the frozen `rsi_canary_readout`, the two supplementary scripts of the confirmatory read, and the engine's own records.

The loop proposed 68 fixes (65 regime, 3 judge). Tier 1 passed 20, found 42 inconclusive and failed 2; Tier 2 passed 8, failed 22, let 3 expire and was never requested for 35 (finding 14). Eight canaries started; five decided inside the cells, four passed and shipped and one failed; three were open at the close (Table 17). Two passes are on fully treated arms: `130b2a03` (`ci-fix` s11, an exception rule for the `pip-audit` pinned-dependency class), 58 graded canary runs, the fix read in all 58, 2 recurred against 0 of 25 in control; and `0a2f5553` (`ci-fix` s23, same class), 25 of 25 read, 0 recurred against 0 of 18. `dd421419` (`ci-fix` s23) passed at 30 graded with 0 recurred against 0 of 39 and the fix read in 9 runs, one above the eight-read floor of amendment (z'); an hour earlier, at 4 reads, the bound was already under the margin and the verdict was held. The fourth pass, `a3da998e` (`support` s11), is a procedure fix ("unresolved handoff loops") graded by the judge's verdicts: 51 graded, read in 18, 7 recurred (14 %) against 9 of 53 (17 %). All four shipped by policy within a second of their verdict. The failure is `7fd04380` (`support` s23): "csv export timeout → split date ranges", the wrong action under set C, where the paid tier's export timeout moved to a settings change. Its canary arm graded 8 runs, read the fix in 7 (4 version-exact), and recurred in 7 (88 %) against 2 of 20 in control (10 %; $p < 0.001$). The loop wrote the failure lesson to `main` within a second, the branch never reached `main`, and the next proposal the repair agent drafted for that class (02:03Z) named the settings action. By the (z'') class rule the failure is decided-untreated, since 7 reads is one short of the floor of 8; by share, 7 of 8 graded runs read it. Probe (b) of the preregistration (inject a harmful fix, watch the gate fail it) was not run; this case arose from the loop's own proposals.

A pass is non-inferiority: the fix did not make the behaviour recur more than the rule it replaced, in the situation it was graded in. The three `ci-fix` passes are on a class the fleet already handled (first-attempt success on `audit-pinned` was 0.89–0.97 before the ships and 0.89–0.96 after); the gate certified no harm there, and because control never recurred either, the bound closed as a pass instead of a futility stop. The support pass needs more care. In its cell the moved `card-declined` class rose from 0.41 (7/17) before the 00:04Z ship to 0.60 (21/35) after. But the other support cell, where nothing shipped, rose from 0.23 to 0.50 over the same clock window, and the procedure the ship merged sits on `main` at confidence 0.25 and was read in 2 of the 254 sessions the cell ran after the ship. The rise is the fleet's own recovery. On the branch the procedure's evidence at the close was 8 successes and 25 failures: the sessions that read a procedure about escalation loops are the sessions already in one, and a per-entry counter cannot tell that from harm, which is why the gate compares arms. The P15′ check on that fix (kept-class sessions that read it) is 1 of 9 against 22 of 41 in control, flagged harmful by the rule at $n = 9$; the cell's kept half was 0.49 before the ship and 0.46 after. So a judge-graded procedure passed a non-inferiority gate on recurrence while its treated kept-class read was poor on nine sessions, and the fleet did not read it after the ship.

Table 17: The eight canaries of the exploratory run (amendment (ab)). Graded, read and recurred are the engine's canary-arm counts at decision or close (rec. = recurred); *exact* is the version-exact read join of the supplementary script; control is graded/recurred. The lever is knowledge except where noted. "Class" is the (z'') rule. Development seeds; exploratory; nothing here enters a confirmatory table.

cell	fix	situation	graded	read	exact	rec.	control	verdict	class
ci-fix s11	`130b2a03`	pip-audit pinned	58	58	57	2	25/0	passed, shipped 01:13Z	treated
ci-fix s11	`dc722d08`	flaky-integration cassette	16	11	11	1	12/0	open	—
ci-fix s23	`dd421419`	pip-audit pinned	30	9	8	0	39/0	passed, shipped 01:16Z	treated
ci-fix s23	`0a2f5553`	pip-audit pinned	25	25	25	0	18/0	passed, shipped 02:13Z	treated
ci-fix s23	`e3a81d36`	pip-audit pinned	7	6	6	0	4/0	open	—
support s11	`a3da998e`	handoff loops (procedure; judge-graded)	51	18	17	7	53/9	passed, shipped 00:04Z	treated
support s11	`a09e0892`	web team export error	21	10	9	4	17/0	open	—
support s23	`7fd04380`	large export timeout	8	7	4	7	20/2	**failed** 01:09Z	untreated ($7 < 8$)

Whether a shipped procedure should inherit a low branch confidence, and whether a judge-graded canary should also carry the situation's kept-class check, are open engine questions.

Finding 14 was found in this run. A repair branch is opened only while the agent has fewer than `MAX_OPEN_BRANCHES` = 10 open, the count included `test_failed`, and a failed fix's branch is never closed. In both support cells nine Tier 1/Tier 2 failures plus the running canary filled the cap (s23 at 23:48Z, s11 at 00:22Z); every later proposal was staged without a branch, and a branchless fix gets no Tier 2 replay and never enters the canary queue. That is the "never requested for 35" above: 29 fixes waited branchless at the close, and the s23 support slot stood idle from the 01:09Z failure to the end with 15 fixes "waiting". The `ci-fix` cells, with at most six branches, were untouched and produced the three treated passes. The fix (amfs-internal #803) reached production at 03:17:37Z, cut `post-branch-cap`, inside the support cells' last thirty episodes; no fix was proposed after it, so the cut has no footprint in these counts. The long horizon on these cells (descriptive; `focus` 1.0, so not comparable to the confirmatory curve) is moved half 40, 73, 73, 70, 82, 78 and 84 % by 50-episode block from episode 50, kept half 69, 66, 78, 79, 76, 68 and 79 %. What this run adds is that the same gate, given the traffic its rule is sized for, decides: it passes treated fixes, holds an untreated one until it is read, and fails a wrong one before it reaches `main`. What it certifies is the absence of harm in the situation; the fleet-level benefit of a ship is still to be separated from the fleet's own recovery.

6 What the loop taught us about itself

The preregistration promised to report every prediction, and Table 18 is that report. The findings below are what a benchmark that knows the true rule found in a shipped self-improvement loop: twelve defects and two properties of the problem. All twelve were fixed in production with their deploy cuts recorded, eleven between runs and re-measured on the same protocol, the twelfth (finding 14) inside the exploratory run of amendment (ab). The last four were found in the fleet-10 runs, two of them from the loop's own read record while the confirmatory fleet-10 cells were running. None of these failures is specific to our engine; the evidence path is where such systems fail.

1. **The loop was never reached.** (Pilots 5b–5g.) Incidents open at three failing verdicts on one behaviour cluster. A fresh agent's judges had graded nothing, so no incident opened and no fix was proposed in 60 episodes. The regime-shift producer, an entry validated ≥ 3 times that now fails, is the producer a memory can drive without judges, and every result here rests on it.

2. **Sealing stalled under load.** A lock on the seal path held sessions open under the grid's concurrency. Fixed (#780/#781) before any reported run.

3. **A first-exposure failure is not a regime shift.** (Amendment (k), pilot 5k; P1 falsified 3/3.) The shift producer fired on entries that had been read, not validated, so the first failure of a never-tested entry opened a canary that could grade nothing: it compared recurrence of a failure the control arm had never had. The guard now requires validation in the situation.

4. **On an unconditional change, demotion recovers almost everything.** (Amendment (l); P4.) A scripted maintainer writing the true rule five episodes after the change beat outcome learning alone by 2.5 (`support`) and 4.2 (`ci-fix`) points on 60 episodes of set A. That is the ceiling a repair loop can reach on such a change, and it is small. Change set C was written so that demotion has no signal: the entry is still right on the kept half.

5. **A regime fix had nothing to be proved against.** (Seam 4, #793; P6 met on the ablation.) Case sets were built from judge verdicts; a fix drafted from a shift had none, so Tier 2 could not be asked. The manifest's runs now seed the case set, and a judge-less case is graded on the replay's outcome.

6. **The agent spells one situation ten ways.** (Seam 5, #794; P7 recall 0.71, purity 0.92 against 0.80/0.90 predicted; seam 5b, #796, P7′ falsified: 0.40/0.67.) A canary scoped to a situation graded 2 of 15 in-situation runs because the agent labelled the same situation `csv export timeout`, `data export error`, `large export failing`. A server-side canonical resolves labels by stemmed tokens. The embedding second stage added for recall lowered it and merged `ci-fix` classes (`format check failure` ∼ `audit check failure` at 0.79); it is left on by decision, with its cost stated.

7. **A running canary swallowed every later fix's replays.** (Seam 7, #795; P8 met.) The router bucketed a Tier 2 replay session like any other, so a replay for fix B sealed during A's canary was stamped onto A's branch, counted in A's arm, and reported `missing` to B. Replays are now re-stamped onto their own branch and excluded from every canary tally.

8. **The loop drafted fixes from its own tests.** (Seam 8, #797; P10 met.) A Tier 2 replay that failed on a fix branch read to the regime pass as the entry failing on `main`. 13 of 40 fixes on the `post-seam5b` cut were drafted from a replay, in chains, and one replay's spelling became a canonical that four served runs then joined. Replays are no longer traffic at the seal: no judge, no identity, no shift. (The first cut of this rule also excluded the labelled copy of a served inference, which is traffic; an automated reviewer caught it before merge.)

9. **The proof graded the wrong signal.** (Amendment (s), #797; P9(b) falsified 20/24.) An outcome-graded case passed a recovered success, so an unconditional flip passed a proof set built to catch it (§4.3). A case now passes only on a clean success, with no recorded failed attempt, and the change is dated by the case, not the deploy, so earlier case sets keep their semantics.

10. **Who grades the proof.** The clean-success rule of finding 9 applied to a case no judge graded. Here 174 of 175 proof cases carry a judge, the benchmark's own frozen rubric, which says by

design that "a run that failed one or more attempts before succeeding is a PASS". The proof tier inherited the coarse signal through the judge as well as through the outcome, and finding 9's fix reached one case in 175. Of the 555 case replays run since the `post-seam5b` cut, 479 passed, and 116 of those passes (24 %) carry a recorded failed attempt. The correction carries the clean-success requirement on every case built for a regime fix, whatever grades it. It shipped as the `post-clean-success` cut, and P11 is read on that cut (§7).

11. **The canary grades by canonical situation, and the agent's declarations fragment.** (Found in the fleet-10 run of §4.4; fixed in #799.) A canary counts exposure (sessions the router stamped to its arm) but graded only runs whose canonical situation equalled the fix's. The agent declares its situation per run in its own words, often carrying the very condition the change turns on ("pip audit requests pin with legacy sdk"); situation identity folded one agent's 56 declarations to 10 canonicals, and the fixes carried others. So 0–8 of 16–30 exposed runs per arm were graded, and no canary decided in 200 episodes with ten agents. The one whose declarations were stable reached 25/30 graded, found 12 % against 17 % recurrence, and timed out, because the rule waited for a significant improvement. Corrected after approval (amendment (v), cut `post-canary-ni`): the canary grades runs in the fix's situation or that read an entry the fix is about, and its primary is non-inferiority, because the canary is the harm gate on a change the replay tiers already proved. Canonical merging was not done; it changes situation identity for every customer with the opposite failure mode. Re-measured (§4.4): graded runs rose from 0–27 % of exposure to 12–55 %, four of nine canaries reached the floor, none decided.

12. **The fix was written where no agent could read it.** (Found 2026-09-30 in the loop's own read record, after the confirmatory fleet-10 cells had run half their length with canary-arm reads of the fix still at zero; fixed in amfs-internal #802, cut `post-canary-visible`.) Every write the repair loop makes (the correction on the fix's branch, the corrective entry, the merged version once a fix ships, the contrast lessons) is authored by the platform itself, as `amfs-server`. The visibility rule that keeps one customer's agents from reading another's admitted an entry only when its author was one of the caller's own agents, or a room member or the platform on a room's shared path. The platform's own writes on an agent's ordinary path matched neither clause, so for every user-scoped API key they did not exist. The reuse record stores, for every retrieve, read and search, which entry version the session was served; joined version-exactly to the arm stamps on the sealed traces, it gave the count. Of 25,875 reuse events over the two days before the fix, none was on a platform-authored entry; no causal entry or read in any canary arm since the `rsi-e2b-c6` cut was the fix's; the 230 contrast lessons on `main` had never been read. On a canary branch the effect was worse than absence: the corrected key's branch version replaced the agent's version of the key and was then dropped, so what the canary arm received was the key's deletion. Every canary in §4.4 compared an arm that had lost the entry with an arm that still had the old one. The rule now admits a platform-authored entry on a path where one of the caller's own agents has written, never account-wide; on the benchmark account that covers 318 of 318 corrections and 168 of 168 corrective entries. One limitation survives: the loop writes a correction at 0.8× the confidence of the version it replaces (floor 0.3) while the benchmark's agents read at a confidence gate of 0.5, so of the 31 shipped fixes that carry a correction, 20 sit below the gate on `main`, visible but not read there; on the canary branch the pin carries the correction to the head regardless. The re-measurement (the four `rsi-conf-e2b` cells, six canaries) was the first time a canary arm read its fix: 17 reads in 23 graded canary-arm sessions for the one fix that rewrote an existing key (`repo-readme-ci`, 74 %), and 0, 0, 1, 1, 1

for the five that wrote a new key. P13″ was falsified (1, 1, 8.5, 0.5) and P16 pooled was 20 of 108 graded sessions, 19.8 %, at the falsification line. That one fix was read and five were not is finding 13.

13. **The loop's own replays discredited the fix before the arm could read it.** (Found 2026-09-30 in the read record of the cells above; fixed in amfs #455, cut `post-replay-evidence`; the confirmatory fleet-10 cells re-run on it, amendment (aa).) Five of the six corrections were discredited by outcome evidence within 1–24 minutes of being written, two before their canary's first session. Traced on `b2b656c5`: the correction was written at 09:00:09Z at confidence 0.78; the loop's three Tier 2 replays ran over the next minute and all ended in `success`, so Tier 2 passed, but two of them carried two recorded attempts each, and each failed attempt charged the entry it had read with a failure. At $n = 0$ one charge takes 0.78 to 0.41; by 09:01:14Z the posterior was under the discredit threshold, 74 seconds before the canary started. The pin of #454 does not pin a discredited entry, so the arm the branch existed to treat was never served it: 0 reads in 74 sessions. `2ae2f0e0` went the same way and then failed its canary on an arm that had never read it (2 recurrences in 8 against 0 in 22); a failing verdict is not read-gated, so the gate rejected a treatment that was never applied. None of the 30 earlier canary corrections had ever been discredited, because nobody could read them; making the fix readable is what exposed this seam. The same replays had written 425 `lesson-contrast-replay-*` entries onto `main` since 2026-09-26, each naming a correction under test as the entry to avoid. The engine already held the rule that a replay is not traffic where the loop grades (finding 8); #455 completes it on the memory side. A replay's outcome writes no outcome row, and a discredited entry is pinned on its own branch all the same, because the canary can only test what the arm read.

14. **The loop's failed drafts used up its right to test new ones.** (Found 2026-10-01 in the exploratory cells of amendment (ab); fixed in amfs-internal #803, cut `post-branch-cap`, 03:17:37Z.) A repair branch is opened only while the agent has fewer than ten open; the count included `test_failed`, and a failed fix's branch is never closed. A long-running agent with a normal Tier 1/Tier 2 failure rate fills its cap with dead branches, as both support cells did within two and a half hours. After that every proposal is staged branchless, gets no Tier 2 replay, and can never enter the canary queue. The loop had degraded to propose-only without saying so: 29 fixes waiting without a branch, a canary slot idle for the last 2.7 hours of a cell with 15 fixes "queued". The fix counts only the statuses the policy can still move. The confirmatory cells were short enough (200 episodes, 4–8 proposals per cell) never to hit the cap, which is why this surfaced only when the horizon doubled.

The canary inside a cell. Separately from the defects, no canary reached a decision inside any 100-episode cell (K1–K4 in the readouts). A situated canary grades only in-situation runs, and at `pct`=50 with a floor of 8 per arm it needs 16 in-situation graded sessions after it starts, which a cell with 4–10 post-change exposures of one class cannot supply. This is a property of the protocol, and the fleet-10 experiments are sized for it.

Table 18: The predictions ledger. Every preregistered prediction on the engine so far, with the amendment that made it, the run that tested it and its verdict. Pending rows are being tested on the run this version reports as open.

P	Where	Prediction (dev seeds, set C unless stated; "conf" = confirmatory seeds)	Verdict
P1	(k) 5k	a shift-drafted fix is graded on runs that read the entry	falsified 3/3
P2	(l) c1	− canary ships a regime fix in every cell; recurrence falls	not met (1/4)
P3	(l) c1	SenseLab's canary reaches a decision in-cell	not met (K1–K4)
P4	(l) 5A	maintainer oracle within 3 pts of outcome learning, set A	+2.5 / +4.2
P5	(l)	judges-only recovers ≥ 0.7 of oracle recovery	pending
P6	(o) c2	every Tier 1-passed regime fix gets a Tier 2 verdict	met (ablation); falsified by seam 7 (SenseLab)
P7	(o) c2	situation recall ≥ 0.80, purity ≥ 0.90	0.71 / 0.92
P7′	(r) c3	embedding stage: recall ≥ 0.80, purity ≥ 0.90	falsified 0.40 / 0.67
P8	(q) c3	replays during a canary are re-stamped and graded	met (26/26)
P9(a,c)	(q) c3	proof set of 3–6 cases; kept-case failures occur	met
P9(b)	(q) c3	$\leq$ 25 % of proof-passing regime fixes non-conditional	falsified 83 %
P10	(s) c5	no fix drafted from a replay; no alias from a replay	met (0 of 47 manifest runs; 134 replays present)
P11	(t) c5	non-conditional share $<$ 50 %; a `ci-fix` kept-case failure; ships fall	(a) falsified 4/4; (b) met, 3; (c) met, 11 → 2
P12	(v) c6	≥ 1 in 2 canaries reach the graded floor on both arms; graded/exposed ≥ 0.8	not met: 4 of 9; graded/exposed 0.12–0.55 (from 0–0.27)
P13	(v) c6	≥ 1 in 3 canaries decide	falsified: 0 of 9 (7 from Tier 1 pass, 2 from inconclusive); 3 of 4 at the floor had 0 recurrence in both arms
P14	(v) c6	no pass harms the kept class by > 10 pp (trace-joined)	undefined: no verdict
P11(a′)	(v) c6	$\geq$ 50 % of kept-case regime drafts state a condition; kept-case rejections $<$ 5 of 23	met, 21 of 37 (author-coded, (y)); CI 11/14 all on the true variable, support 9/23 none on it; rejections 4 of 17 and 11 of 39; the count fell, the share did not
(x) primary	conf f1	SenseLab − each baseline > 0 on the moved half; exact sign-flip over 16 cells, Holm across the five	met in sign and in cells won: +19/+4/+13/+9/+13 pp, 11/8/11/9/10 of 16; p 0.024/0.544/0.040/0.119/0.040, Holm 0.12/0.54/0.16/0.24/0.16; none survives
(x) secondary	conf f1	kept half and unchanged classes, same test, Holm within endpoint	kept: +38 vs Frontier (Holm 0.007), −24 vs the context window (1 of 16 cells, Holm 0.001), level with the pgvector arms; unchanged: +23/+9/+16 vs Frontier/rerank/DIY (Holm 0.002/0.035/0.004), +3 and +4 vs the context window and pgvector (Holm 0.13, 0.06)
P11(a′)	conf f1	$\geq$ 50 % of kept-case regime drafts state a condition	met, 27 of 39 (author-coded, same sheet and key); `ci-fix` 20 of 28, 17 of them on the PR's files; `support` 7 of 11, none on the true variable

P	Where	Prediction (continued)	Verdict
P12–P14, P13′, P13″, P15′, P16	(z″) e2b, finding 13	as (v), (z), (z′), (z″) on the fleet-10 cells run on `post-canary-visible`	measured, not the confirmatory read (five of six corrections discredited by replays before the arm read them): P12 s59 2 of 2 and s73 1 of 2 at the floor; P13 1 of 6; a failing verdict on an arm that never read the fix; P14 undefined; P13″ falsified, 1/1/8.5/0.5; P15′ no population; P16 20 of 108 graded sessions (19.8 %), 74 % on the one fix that was read
same	conf e2b-r2	as above, on `post-replay-evidence` (amendment (aa)); the confirmatory fleet-10 read	P12 met, 4 of 6 at the floor on both arms (graded/exposed 0.23–0.36 on five, 1.0 on one: second clause not met); P13′ not met, 1 of 6 decided (failed, zero-count bound, arm read the fix in 5 of 10 graded sessions; below the read floor, so untreated by the class rule); P14/P15/P15′ no population (0 passes; treated kept-class comparisons all under 8 sessions); P13″ met, reads at close 2/5/8/12/14/18, median 10 vs floor 8; P16 met, 57 of 76 graded canary-arm sessions (75 %), control 0 of 54; 0 shipped, 0 reverted

7 Pending, and what we expect

Every preregistered prediction and its verdict is in Table 18; this section states what the verdicts changed. P10 (no fix drafted from a replay; no alias from a replay) is met on `rsi-c5`: none of the 47 runs cited by 33 fix manifests carries a replay tag, while 134 of the 928 sessions sealed in the same window were replays. P11(a), that fewer than half of proof-passing regime fixes would omit the conditional, was falsified 4 of 4, and the falsification located where the conditional is lost: in the repair agent's draft, since the proof removed 19 of 23 regime fixes and the four survivors were wholesale flips whose kept-half replays passed because the replaying agent did not follow the entry. The drafting change that followed (amendment (v)) is read by P11(a′), met on both the development runs (21 of 37) and the confirmatory fleet-1 runs (27 of 39), with the class split in §4.3. P12–P14 were first read on `rsi-e2b-c6` (P12 not met, P13 falsified, P14 undefined), and finding 12 then showed that those canaries compared two untreated arms. The fleet-10 confirmatory cells ran on `post-canary-visible` with the read-count predictions of amendments (z), (z′) and (z″) pre-specified; that run exposed finding 13 and is reported as measured. The confirmatory read is the rerun on `post-replay-evidence` (§5): the arm reads its fix (P16, P13″ met), four of six canaries reach the floor on both arms (P12 met), one decides (P13′ not met), nothing ships (P14, P15 without population). The exploratory run of amendment (ab) is the only place in this paper where a canary passes and a fix ships through it, and it bears on no prediction.

The confirmatory scope, reduced before it ran. Amendment (w), declared before any confirmatory-seed cell and before the analysis freeze, set the confirmatory grid at seeds 37/41/59/73 on the two set-C families, both agent models, fleet 1 with every system and both ablations, and fleet 10 with SenseLab and its two gate ablations: 22,400 episodes. Amendment (y), declared the morning the runs launched, cut the fleet-10 part to SenseLab alone on GPT-5.5 after the second development fleet-10 run read the ablations the same way as the first, and replaced the two blind coders with the author. The fleet-10 ablation comparison therefore has no confirmatory read. The fleet-10 cells ran twice, after finding 12 and after finding 13 (amendment (aa), declared before the rerun launched): 16,000 confirmatory episodes in all. The loop has not been run on the remaining eight families; their set C is unwritten, and the held-out-family design of amendment (l) lapses with it. The mixed-effects learning-curve model named in the preregistration is not run, for the reasons

given there. Not run at all, and reported as not run: verdict sources (E3), the public track (E4), the A/A, adversarial, reward-hacking and manifest-calibration probes, and the Managed Model rungs above L0 (E7). Each is a claim this paper does not make. The adversarial probe (b) has no injection path in the harness; the one wrong fix the gate failed arose from the loop's own proposals and is not a substitute for it.

8 Related work

Self-improvement. Self-Refine [9] and STaR [10] improve a model's outputs or weights from its own feedback within a task or a training run. The Gödel machine lineage [11, 12] and AlphaEvolve [13] modify the improver itself, gated by a proof or a benchmark score, in a sandbox. The engine here is the deployed case: the object it modifies is what every live agent acts on, the gate is live traffic, and the contribution is the discipline (manifest, tiers of proof, staged exposure, reversibility, audit trail) rather than the search. The "fixed budget, held-out, sustained, generalising" conditions the preregistration adopts for any claim of self-improvement (E5) are the four conditions Weco proposes for the "net positive" rung of its ladder [17] and reports meeting with AIDE^2 [18]. The ladder's rungs number the strength of a claim; the levels of this paper number the stages of one loop. E5 has not run, and no rung is claimed.

Release engineering. Canary release and controlled online experiments [14] are the discipline the loop borrows for level 1. A/A tests and look-schedule inflation are the standard hazards, and the preregistration's A/A probe measures the empirical false-pass rate of the canary's test under the schedule the harness uses.

Benchmarks. τ-bench and τ^2-bench [15, 16] evaluate tool-using agents against a policy that stays fixed across tasks; our public track (E4) amends the policy between passes. Continual-learning benchmarks for language models measure forgetting in weights. The object here is what a deployed agent acts on, and the failure mode is remembering.

Agent memory. Reflexion [2] and Voyager [3] write verbal reflections and skills back for later episodes; Generative Agents [4] and MemGPT [5] manage a store with retrieval and summarisation; Mem0 [6] and Zep [7] are production memory layers evaluated on conversational recall (LoCoMo, LongMemEval [8]). All are evaluated on retrieving what was stored. None tests, gates or reverts a change to the store, and none is measured on a change of the world after storage. They are the substrate a self-improving engine operates on and the baselines it is measured against here; our earlier report [1] measured level 0 against these designs under a regime change.

9 Threats to validity

- **Development seeds.** Every number in §4 is from two seeds on two families, chosen for development. Nothing there is a confirmed difference, and the intervals say so.
- **One model, one effort level, on the development seeds.** GPT-5.5 at low effort. Claude Opus 5 is half of the confirmatory grid, where every cell is model-aware.
- **The benchmark's outcome is coarse.** The harness reports `success` for an episode solved within three attempts. That is what a customer reports, and it hid finding 9 from the proof set

for two runs. The clean-success rule reads the attempts the agent already records; a customer whose agent never records attempts gets no protection from it.

- **The engine changed between runs.** Twelve defects were fixed in production during the study (amfs-internal #780–#803, OSS #454–#455): one inside the fleet-1 confirmatory window, two between the two fleet-10 confirmatory runs, and one (#803) inside the last thirty episodes of the exploratory cells, after which no fix was proposed. Every result carries its cut. The changes are to the loop's evidence path, not to any threshold constant, which the preregistration forbids. The one gate rule that changed, the canary's primary from superiority to non-inferiority (finding 11), was preregistered with its falsifier before the run that tests it.
- **Hand-coding.** The "conditional" share (P9(b), P11, P11(a′)) is one coder's reading of the draft text against written instructions, and the coder is the author, not blind to arm or verdict (amendment (y)). Every draft, the instructions, the coding and the key are published so the reading can be redone.
- **The harness routes the canary.** The production router runs in the MCP gateway; the harness applies the same hash itself (parity-tested in CI) because it talks to the HTTP API.
- **The context window is a strong baseline where it fits.** On `ci-fix` the whole cell fits in 24k tokens and the window sees the change. It is the right comparison for short horizons and unavailable for the 200-episode fleet, where it cannot fit.
- **We built the system.** The preregistration, the amendments with their falsifiers, the cut records and the ledger exist so that a reader can check what was predicted before each run.

A Reproducibility

Runs. Harness arm names, cell ids and command lines: baselines `rsi-c1-tier1` (`none`, `pgvector+rerank`, `senselab`), `rsi-c1-baselines` (`pgvector`, `pgvector-diy+outcomes`), `rsi-c1-baselines-cw` (`raw-traces`); SenseLab `rsi-c1-rsi`, `rsi-c2`, `rsi-c3`, `rsi-c5`, `rsi-c6` (`senselab-rsi`, `senselab-rsi-nocanary`); fleet 10 `rsi-e2b-c5`, `rsi-e2b-c6` (plus `senselab-rsi-noreplay`, 200 episodes, change at 50, `-rep`); confirmatory `rsi-conf-f1-s<seed>`, `rsi-conf-e2b-s<seed>` (the `post-canary-visible` run, finding 13) and `rsi-conf-e2b-r2-s<seed>` (the `post-replay-evidence` rerun; `CL_CONF_RUN_TAG=r2`) from `confirm_rsi.sh`; exploratory `rsi-x-canary-s11`, `rsi-x-canary-s23` (amendment (ab): `--episodes 400 --change-focus 1.0`, `CL_RSI_CANARY_TIMEOUT=400`, `CL_RSI_CANARY_WINDOW=400`, otherwise the e2b line). Cell id `<family>|<arm>|gpt-5.5|s<seed>|e100-d300-n0.0-f0-a1-c30-x0.5-oid-vC`. Command:

```
python -m continual_learning.runner --run <run> --scenarios support ci-fix --arms <arms> --models
gpt-5.5 --seeds 11 23 --fleet 1 --episodes 100 --change-at 30 --change-focus 0.5 --change-variant
C --opaque-ids --workers 2
```

with `CL_RSI_TIER2=1` and a Tier 2 replay receiver reachable from the service.

Engine cuts. `pre-seam4`: amfs-internal #787/#788. `post-seam5`: #793 (07:00:22Z), #794 (08:27:24Z, 2026-09-27). `post-seam5b`: #795 (12:50:24Z), #796 (13:39:13Z). `post-seam8`: #797 (09:14:17Z, 2026-09-28). `post-clean-success`: #798. `post-canary-ni`: #799 (11:35:32Z, 2026-09-29). `post-canary-zero`: OSS #454, amfs-internal #800/#801 (03:40:19Z, 2026-09-30). `post-canary-visible`: #802 (08:21:05Z, 2026-09-30). `post-replay-evidence`: OSS #455

(12:52:04Z, 2026-09-30; the confirmatory fleet-10 cut). `post-branch-cap`: #803 (03:17:37Z, 2026-10-01; inside the last episodes of the exploratory cells, no fix proposed after it). Each cut's Cloud Run revision is in `HYPOTHESIS_RSI.md`.

Analysis. `python -m benchmarks.continual_learning.analysis.rsi --paper` writes `results/rsi-report/REPORT.md` and `report.json` for the development seeds; `--seeds 37 41 59 73 --models gpt-5.5 claude-opus-5 --runs <confirmatory runs> --report-dir results/rsi-report-confirmatory` is the confirmatory read, run on the staged copy of the fleet-1 results (one run per system; the staging script and its byte-identity check are in the package). `analysis.rsi_canary_readout --run <run>` is the per-canary read record (verdicts, reads by class of run, the version-exact join); the three-class canary-arm read count of findings 12–13 is a supplementary script in the package that reads the same tables. `analysis.rsi_coding_sheet make --runs <runs>` and `score --answers` produce and score the P11(a′) sheets. `analysis.fill_rsi_paper` fills this document from the report and `analysis.rsi_figures` draws the figures. The analysis code is tagged `analysis-freeze-rsi` before the confirmatory runs. The preregistration, amendments, readouts (`HEADROOM_RSI.md`), change sets, run records and coding sheets (drafts, instructions, author's coding, key) are published with the paper; the supplementary scripts, the confirmatory and exploratory readouts and the coding sheets are in `paper/package/` on the paper branch.

References

[1] SenseLab Research. From memory to recursive self-improvement: agents that learn from outcomes, agents that only remember, and a model trained on what they learned, when the tasks never repeat and the rules change. Preprint, 2026.

[2] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023.

[3] G. Wang et al. Voyager: An open-ended embodied agent with large language models. TMLR 2024 (arXiv:2305.16291).

[4] J. S. Park et al. Generative agents: Interactive simulacra of human behavior. UIST 2023.

[5] C. Packer et al. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.

[6] P. Chhikara et al. Mem0: Building production-ready AI agents with scalable long-term memory. arXiv:2504.19413, 2025.

[7] P. Rasmussen et al. Zep: A temporal knowledge graph architecture for agent memory. arXiv:2501.13956, 2025.

[8] D. Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. ICLR 2025.

[9] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS 2023.

[10] E. Zelikman, Y. Wu, J. Mu, N. D. Goodman. STaR: Bootstrapping reasoning with reasoning. NeurIPS 2022.

[11] J. Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers. In *Artificial General Intelligence*, Springer, 2007.

[12] J. Zhang, S. Hu, C. Lu, R. Lange, J. Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. arXiv:2505.22954, 2025.

[13] A. Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131, 2025.

[14] R. Kohavi, D. Tang, Y. Xu. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press, 2020.

[15] S. Yao, N. Shinn, P. Razavi, K. Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045, 2024.

[16] V. Barres et al. τ^2-bench: Evaluating conversational agents in a dual-control environment. arXiv:2506.07982, 2025.

[17] Weco AI. 4 levels of recursive self-improvement. Blog post, July 2026. `https://www.weco.ai/blog/4-levels-of-recursive-self-improvement`.

[18] Z. Jiang et al. AIDE^2: First evidence of recursive self-improvement. Weco AI blog post, July 2026. `https://www.weco.ai/blog/first-evidence-of-recursive-self-improvement`.